

Atelier 3 : la base de la clinique, en SQL

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Base de données 1, fondements, modélisation et SQL · **Chapitre** : 3, Créer et manipuler les données en SQL · **Version** : 2026-10-31

Avant de commencer

Ce que vous allez construire	Le script SQL complet de la base de la clinique Santé Plus : les **dix tables** de votre MLD (atelier 2) avec toutes leurs contraintes, les données, des mises à jour et une transaction d'hospitalisation
Durée	🟢 Débutant : 7 h · 🔵 Standard : 5 h · 🟣 Expert : 3 h 30 (sans les défis)
Prérequis	Avoir lu le chapitre 3 ; l'atelier 2 terminé (votre MLD)
Fichiers fournis	`atelier3-donnees-depart.sql` : les données des six tables du chapitre (services, médecins, patients, consultations, médicaments, prescriptions)
Rendu	Le fichier `clinique.sql` (création + données, rejouable) et le fichier `atelier3-suite.sql` (étapes 5 à 7), avec en commentaires les messages d'erreur obtenus

A la fin, votre base contient :

Table	Lignes	Table	Lignes
`service`	4	`medicament`	6
`medecin`	6	`prescription`	10
`assurance`	3	`chambre`	5
`patient`	8	`sejour`	3
`consultation`	10	`paiement`	9

> 📍 **Point de départ : votre MLD de l'atelier 2.** Pour que tout le monde ait la même base à l'atelier 4, utilisez **exactement** les noms de tables et de colonnes du chapitre 3 pour les six premières tables, et ceux-ci pour les quatre nouvelles :
>
> - **assurance** (code_assurance, nom_assureur, taux_prise_en_charge)
> - **patient** gagne la colonne #code_assurance
> - **chambre** (num_chambre, type_chambre, prix_nuit, etage_chambre, #code_service)
> - **sejour** (num_sejour, date_entree, date_sortie, #num_patient, #num_chambre, #num_medecin)
> - **paiement** (num_paiement, date_paiement, montant, mode_paiement, #num_consultation)
>
> **La suite du fil rouge** : atelier 3 (ce travail) → atelier 4 (les requêtes de la clinique, sur cette base).

Comment travailler

L'atelier est découpé en **9 étapes** (0 à 8). Écrivez tout dans des fichiers `.sql` et exécutez-les **en entier** régulièrement : un script qui ne se rejoue pas n'est pas fini.

Trois règles pour tout l'atelier

1. **Toutes les contraintes sont nommées** (`CONSTRAINT pk_...` , `fk_...` , `ck_...`).
2. **Pas de** `UPDATE` **ni de** `DELETE` **sans** `WHERE` , et pas sans avoir d'abord vu les lignes concernées avec un `SELECT` .
3. **Notez chaque message d'erreur** en commentaire (`-- ERREUR obtenue : ...`) au-dessus de l'instruction qui l'a provoqué.

Étape 0 : préparer la base (10 min)

🎯 **Objectif** : repartir d'une base vide.

🛠️ **A faire**

1. Dans `clinique` , supprimez les tables des chapitres précédents (enfants d'abord).
2. Créez le fichier `clinique.sql` , avec en tête un commentaire : votre nom, votre SGBD, la date.

✅ **Vérification** : la base `clinique` ne contient plus aucune table.

Étape 1 : un script qui se rejoue (20 min)

🎯 **Objectif** : pouvoir exécuter `clinique.sql` autant de fois qu'on veut.

🛠️ **A faire**

1. Écrivez, en haut de `clinique.sql` , un `DROP TABLE IF EXISTS` pour chacune des dix tables, **dans l'ordre inverse** de leur création.

2. En commentaire, écrivez l'ordre de création des dix tables (parents d'abord).

🔎 **A chercher** : chapitre 3, sections 2.2 et 4.2 ; « SQL DROP TABLE IF EXISTS ».

💡 **A comprendre**

- Pourquoi `paiement` doit-elle être supprimée **avant** `consultation` ? Et `patient` avant `assurance` ?
- Que fait `IF EXISTS` la toute première fois, quand les tables n'existent pas encore ?

✅ **Vérification** : exécuté sur une base vide, le début du script ne provoque aucune erreur.

<details>
<summary>▼ Indice : un ordre de création possible</summary>

`service`, `medecin`, `patient`, `consultation`, `medicament`, `prescription`, `assurance`, `chambre`, `sejour`, `paiement`. Pour supprimer : l'ordre inverse. (Créer `assurance` juste avant `patient` marche aussi : la clé étrangère peut alors être écrite directement dans le `CREATE TABLE patient`.)
</details>

Étape 2 : les six tables du chapitre (30 min)

🎯 **Objectif** : reprendre les tables du chapitre 3, avec toutes les contraintes nommées.

🛠️ **A faire**

1. Recopiez (en les **tapant**) les six `CREATE TABLE` du chapitre 3 (section 2.1), en **nommant** chaque contrainte `CHECK`.
2. Ajoutez à `patient` la colonne `code_assurance VARCHAR(10)`, **sans** clé étrangère pour l'instant : la table `assurance` n'existe pas encore. Vous ajouterez la clé étrangère à l'étape 3, avec `ALTER TABLE`.
3. Donnez à la clé étrangère `fk_prescription_consultation` le comportement `ON DELETE CASCADE`.

🔎 **A chercher** : chapitre 3, sections 2 et 7.

✅ **Vérification** : les six tables existent ; la description de `patient` (`DESCRIBE patient;` sur MySQL, `\d patient` dans `psql`, ou le panneau de votre outil) montre 8 colonnes.

Étape 3 : les quatre nouvelles tables (45 min)

🎯 **Objectif** : traduire en SQL ce que vous avez conçu à l'atelier 2.

🛠️ **A faire** : créez `assurance`, `chambre`, `sejour` et `paiement` avec ces règles.

Table	Règles à traduire en contraintes
`assurance`	Code de 10 caractères au plus ; nom obligatoire ; taux obligatoire, entier **entre 0 et 100**

Table	Règles à traduire en contraintes
`patient`	Après avoir créé `assurance` : ajoutez avec `ALTER TABLE patient ADD CONSTRAINT ...` la clé étrangère `fk_patient_assurance`, **facultative** ; si une assurance est supprimée, ses patients n'ont plus d'assurance (`ON DELETE SET NULL`)
`chambre`	Numéro de 6 caractères (`P-12`) ; type `individuelle` **ou** `double` ; prix par nuit **strictement positif** ; étage obligatoire ; service obligatoire
`sejour`	Date d'entrée obligatoire ; date de sortie **facultative**, mais si elle est donnée, elle n'est **pas avant** la date d'entrée ; patient, chambre et médecin obligatoires
`paiement`	Date et montant obligatoires, montant **strictement positif** ; mode parmi `especes` , `mobile money` , `carte` , `assurance` (écrits ainsi, sans accent) ; consultation obligatoire

🔎 **A chercher** : chapitre 3, sections 2 et 7 ; pour la date de sortie, combinez `IS NULL` et `OR` dans le `CHECK` .

💡 **A comprendre**

- Pourquoi `date_sortie` peut-elle être `NULL` ? Que veut dire un séjour dont la date de sortie est `NULL` ?
- Pourquoi `SET NULL` pour l'assurance, mais **pas** pour la consultation d'un paiement ?

✅ **Vérification** : `clinique.sql` s'exécute en entier, deux fois de suite, sans erreur ; la base a dix tables.

▼ Indice : le CHECK des dates de séjour

`CONSTRAINT ck_dates_sejour CHECK (date_sortie IS NULL OR date_sortie >= date_entree)` : « soit pas encore de sortie, soit une sortie après (ou le jour de) l'entrée ».

Étape 4 : les données (45 min)

🎯 **Objectif** : remplir les dix tables.

🛠️ **A faire**

1. Ajoutez à la fin de `clinique.sql` le contenu du fichier fourni `atelier3-donnees-depart.sql` . Exécutez : les six tables du chapitre sont remplies.

2. Insérez les trois assurances, puis donnez leur assurance aux patients avec des `UPDATE` :

code_assurance	nom_assureur	taux	Patients assurés
`IPM-SONA`	IPM Sonatel	80	n°1 (Awa Diop), n°5 (Mariama Sy)
`MUT-ENS`	Mutuelle des enseignants	70	n°2 (Moussa Ndiaye)
`ASS-PRIV`	Assurance privée Teranga	100	n°7 (Khady Thiam)

3. Insérez les cinq chambres et les trois séjours :

num_chambre	type	prix_nuit	étage	service
`P-11`	individuelle	25000	1	PED
`P-12`	double	15000	1	PED
`C-21`	individuelle	35000	2	CAR
`M-13`	double	20000	1	MAT
`G-01`	individuelle	20000	0	GEN

num_sejour	entrée	sortie	patient	chambre	médecin
1	2026-10-15	2026-10-18	6	P-12	4
2	2026-10-19	2026-10-24	7	C-21	3
3	2026-10-27	(encore hospitalisée)	5	M-13	5

4. Insérez les neuf paiements :

num	date	montant	mode	consultation
1	2026-10-05	10000	especes	1
2	2026-10-05	15000	mobile money	2
3	2026-10-12	14000	assurance	3
4	2026-10-12	6000	especes	3
5	2026-10-14	12000	assurance	4
6	2026-10-15	15000	mobile money	5
7	2026-10-19	20000	assurance	6
8	2026-10-20	5000	especes	7
9	2026-10-21	8000	assurance	8

🔎 **A chercher** : chapitre 3, sections 3 et 5 ; « SQL IN » pour donner la même assurance à deux patients en un seul `UPDATE`.

💡 A comprendre

- Pourquoi le fichier fourni insère-t-il les patients **sans** leur assurance ? (Pensez à l'ordre des tables.)
- Le paiement n°3 (14000) et le n°4 (6000) concernent la même consultation, de 20000 FCFA. Comment ces montants s'expliquent-ils, avec l'assurance de Moussa Ndiaye ?

✅ **Vérification** : les nombres de lignes du tableau en début d'énoncé ; `SELECT num_patient, code_assurance FROM patient ORDER BY num_patient;` donne `IPM-SONA`, `MUT-ENS`, `NULL`, `NULL`, `IPM-SONA`, `NULL`, `ASS-PRIV`, `NULL`.

Étape 5 : mettre les contraintes à l'épreuve (30 min)

🎯 **Objectif** : vérifier que chaque règle est vraiment protégée par le SGBD.

🛠️ **A faire** : dans un nouveau fichier `atelier3-suite.sql`, essayez chacune de ces opérations, qui **doivent toutes être refusées**. Notez le message obtenu et le nom de la contrainte qui a refusé.

1. Une chambre `P-14` de type `suite`.
2. Un séjour qui sort le 18 octobre alors qu'il est entré le 20.
3. Un paiement de 0 FCFA.
4. Une assurance qui rembourse 120 %.
5. Un séjour dans la chambre `X-99`, qui n'existe pas.
6. La suppression de la consultation n°8, qui a été (en partie) payée.

💡 **A comprendre**

- Pour l'opération 6, pourquoi le `CASCADE` de `prescription` n'a-t-il pas suffi à supprimer la consultation ?

✅ **Vérification** : six refus. Par exemple, l'opération 1 est refusée par `ck_type_chambre` et l'opération 6 par `fk_paiement_consultation`. Si l'une **passe**, il manque une contrainte : corrigez `clinique.sql`, rejouez-le, et recommencez.

Étape 6 : la vie de la clinique (30 min)

🎯 **Objectif** : modifier des données en toute sécurité.

🛠️ **A faire** (dans `atelier3-suite.sql`, chaque fois précédé du `SELECT` qui montre les lignes concernées)

1. Le prix des chambres **individuelles** augmente de 5000 FCFA.
2. Mariama Sy (séjour n°3) sort le 30 octobre 2026.
3. Testez le `SET NULL` : créez une assurance `TEST` (taux 50), donnez-la au patient n°8, supprimez l'assurance, puis vérifiez l'assurance du patient n°8.
4. La consultation n°10 (paludisme d'Aliou Ndiaye) a été saisie par erreur. Supprimez-la, et vérifiez ce que sont devenues ses prescriptions.

🔎 **A chercher** : chapitre 3, sections 5 et 7.

✅ **Vérification** :

Contrôle	Résultat attendu
Prix des chambres (`ORDER BY num_chambre`)	C-21 : 40000 ; G-01 : 25000 ; M-13 : 20000 ; P-11 : 30000 ; P-12 : 15000
Séjour n°3	date de sortie 2026-10-30

Contrôle	Résultat attendu
Patient n°8 après suppression de `TEST`	`code_assurance` vaut `NULL`
Prescriptions de la consultation n°10	0 (supprimées en cascade)

Étape 7 : la transaction d'hospitalisation (35 min)

🎯 **Objectif** : enregistrer un événement en plusieurs instructions, qui doivent réussir ensemble.

🛠️ **A faire** : Ibrahima Fall (patient n°4) arrive le 28 octobre 2026 avec un paludisme grave. Dans **une seule transaction** :

1. la consultation n°11 (motif `Paludisme grave`, tarif 10000, médecin n°6) ;
2. le séjour n°4, en chambre `G-01`, suivi par le médecin n°6, sans date de sortie ;
3. un point de sauvegarde `avant_paiement` ;
4. un premier essai de paiement n°10 de 10000 FCFA par **chèque** (refusé : ce mode n'existe pas) ;
5. le retour au point de sauvegarde, puis le paiement n°10 en **mobile money** ;
6. la validation.

🔎 **A chercher** : chapitre 3, section 8.

💡 **A comprendre**

- Sur PostgreSQL, que se passerait-il si vous oubliiez le `ROLLBACK TO SAVEPOINT` après l'erreur ? Et sur MySQL ?
- Pourquoi la consultation et le séjour n'ont-ils **pas** été perdus, malgré l'erreur du paiement ?

✅ **Vérification** : 4 séjours, dont le n°4 (patient 4, chambre G-01, sans date de sortie) ; un seul paiement pour la consultation 11 : 10000 en `mobile money`.

Étape 8 : rejouer et rendre (15 min)

🎯 **Objectif** : un rendu propre.

🛠️ **A faire**

1. Exécutez `clinique.sql` en entier : vous retrouvez la base de l'étape 4 (les modifications des étapes 6 et 7 sont dans `atelier3-suite.sql`).
2. Exécutez ensuite `atelier3-suite.sql` en entier (les instructions qui doivent échouer font des erreurs, c'est normal) : vous retrouvez les résultats des étapes 6 et 7.
3. Rendez les deux fichiers.

✅ **Vérification** : les deux scripts s'exécutent dans cet ordre, sur une base vide, avec pour seules erreurs celles de l'étape 5 et le paiement par chèque.

🏆 Défis (facultatifs)

1. **Numérotation automatique.** Réécrivez `paiement` avec un numéro automatique (`AUTO_INCREMENT` ou `GENERATED BY DEFAULT AS IDENTITY`) et insérez les paiements **sans** donner leur numéro.
2. **Un autre SGBD.** Exécutez `clinique.sql` sur FreeSQL (Oracle). Qu'avez-vous dû changer ? (Pensez à `DROP TABLE IF EXISTS` , qui n'existe sur Oracle qu'à partir de la version 23.)
3. **La chambre double pleine.** Imaginez une requête qui compte les patients **actuellement** présents dans une chambre (séjours sans date de sortie). Vous l'écrirez pour de vrai au chapitre 4.
4. **Le médecin du séjour.** Ajoutez une contrainte qui vérifie que le médecin d'un séjour est bien dans le même service que la chambre. Est-ce possible avec un `CHECK` ? Cherchez pourquoi (indice : un `CHECK` ne regarde qu'une ligne d'une table).

Grille d'autoévaluation

Je sais…	Oui	Pas encore
traduire un MLD en `CREATE TABLE` , avec les types adaptés	☐	☐
écrire et nommer toutes les contraintes (clés, `NOT NULL` , `UNIQUE` , `CHECK` , `DEFAULT`)	☐	☐
choisir le comportement d'une clé étrangère à la suppression (refus, `CASCADE` , `SET NULL`)	☐	☐
créer les tables et insérer les données dans le bon ordre	☐	☐
écrire des `UPDATE` et `DELETE` sûrs, vérifiés par un `SELECT`	☐	☐
enregistrer plusieurs opérations dans une transaction, avec un point de sauvegarde	☐	☐
écrire un script qui se rejoue sur une base vide	☐	☐